

Step 2 — A Real ReAct Agent That Calls a Discovered Tool — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 1's `example_server.py` (with `get_weather` and `add_numbers`) and Step 1's `list_server_tools()` from `mcp_connection.py`.

Basic Version

Approach 1 — the direct way, connection kept open for the whole run

```
# tool_bridge.py
import json

def mcp_tools_to_openai_schema(tools):
    result = []
    for tool in tools:
        result.append({
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or "",
                "parameters": tool.inputSchema,
            },
        })
    return result

async def call_mcp_tool(session, name, arguments):
    result = await session.call_tool(name, arguments)
    parts = []
    for block in result.content:
        if hasattr(block, "text"):
            parts.append(block.text)
    return " ".join(parts)
```

```
# agent.py
from openai import OpenAI
from tool_bridge import call_mcp_tool

client = OpenAI()

async def run_agent(task, session, openai_tools):
    messages = [{"role": "user", "content": task}]

    while True:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=openai_tools,
        )
        message = response.choices[0].message

        if not message.tool_calls:
            return message.content

        messages.append(message)
        for tool_call in message.tool_calls:
            name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)
            result_text = await call_mcp_tool(session, name, arguments)
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": result_text,
            })
```

```
# main.py
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from tool_bridge import mcp_tools_to_openai_schema
from agent import run_agent

async def main():
    server_params = StdioServerParameters(command="python", args=["example_server"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
```

```
openai_tools = mcp_tools_to_openai_schema(tools_result.tools)

answer = await run_agent("What's the weather in Paris, FR?", session,
print(answer)

asyncio.run(main())
```

Expected output: something like `The weather in Paris, FR is 18C and cloudy.`

This works correctly, and it makes the one connection-lifetime decision that matters: the `ClientSession` stays open for the entire agent run, opened once in `main()` and passed down into `run_agent`, instead of reconnecting per tool call. It has `while True` with no hard stop, no logging, and no handling of a tool's own `isError` flag — all fine for a first working version, all fixed below.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, a real `max_iterations` limit, and a step log

```
# tool_bridge.py
from mcp.types import Tool

def mcp_tools_to_openai_schema(tools: list[Tool]) -> list[dict]:
    return [
        {
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or "",
                "parameters": tool.inputSchema,
            },
        }
        for tool in tools
    ]

async def call_mcp_tool(session, name: str, arguments: dict) -> str:
    result = await session.call_tool(name, arguments)
    parts = [block.text for block in result.content if hasattr(block, "text")]
    return " ".join(parts) if parts else "(tool returned no text content)"
```

```
# agent.py
import json
from openai import OpenAI
from tool_bridge import call_mcp_tool

client = OpenAI()

async def run_agent(task: str, session, openai_tools: list[dict], max_iterations:
    messages = [
        {"role": "system", "content": "You are a helpful assistant. Use the avail
        {"role": "user", "content": task},
    ]
```

```

log: list[dict] = []

for step in range(max_iterations):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        tools=openai_tools,
    )
    message = response.choices[0].message

    if not message.tool_calls:
        log.append({"step": step, "type": "final_answer", "content": message.
        return {"answer": message.content, "log": log}

    messages.append(message)
    for tool_call in message.tool_calls:
        name = tool_call.function.name
        arguments = json.loads(tool_call.function.arguments)
        log.append({"step": step, "type": "tool_call", "tool": name, "argumen

        result_text = await call_mcp_tool(session, name, arguments)
        log.append({"step": step, "type": "observation", "tool": name, "resul

        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result_text,
        })

    raise RuntimeError(f"Agent did not finish within {max_iterations} steps. Log:

```

Expected output, printing `result["log"]` after a run: a list showing each `tool_call` paired with its `observation`, ending in one `final_answer` entry — the full scratchpad, not just the final string.

Difference from Basic: a real, typed `max_iterations` cap instead of `while True` — Doc07's "a limit isn't optional" applied for real. Every step is logged as it happens (call and result both recorded), and the return value is a small dict carrying both the answer and the full log, instead of just a bare string — Step 4's testing needs that log to confirm *what actually happened* during a run, not just whether the final answer looked right.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — handles `isError`, a missing description, and a named `MaxIterationsExceeded`

```
# tool_bridge.py
import logging
from mcp.types import Tool

logger = logging.getLogger(__name__)

def mcp_tools_to_openai_schema(tools: list[Tool]) -> list[dict]:
    schemas = []
    for tool in tools:
        if not tool.description:
            logger.warning("Tool '%s' has no description -- the model will struggle")
        schemas.append({
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or f"(no description provided for {tool.name})",
                "parameters": tool.inputSchema,
            },
        })
    return schemas

async def call_mcp_tool(session, name: str, arguments: dict) -> str:
    result = await session.call_tool(name, arguments)
    parts = [block.text for block in result.content if hasattr(block, "text")]
    text = " ".join(parts) if parts else "(tool returned no text content)"
    if result.isError:
        return f"Error from tool '{name}': {text}"
    return text
```

```
# agent.py
import json
from openai import OpenAI
from tool_bridge import call_mcp_tool
```

```

client = OpenAI()

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, log: list[dict]):
        super().__init__(message)
        self.log = log

async def run_agent(task: str, session, openai_tools: list[dict], max_iterations:
    messages = [
        {"role": "system", "content": "You are a helpful assistant. Use the avail
        {"role": "user", "content": task},
    ]
    log: list[dict] = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=openai_tools,
        )
        message = response.choices[0].message

        if not message.tool_calls:
            log.append({"step": step, "type": "final_answer", "content": message.
            return {"answer": message.content, "log": log}

        messages.append(message)
        for tool_call in message.tool_calls:
            name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)
            log.append({"step": step, "type": "tool_call", "tool": name, "argumen

            result_text = await call_mcp_tool(session, name, arguments)
            log.append({"step": step, "type": "observation", "tool": name, "resul

            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": result_text,
            })

    raise MaxIterationsExceeded(f"Agent did not finish within {max_iterations} st

```

Expected output, on a task where a tool call is given a deliberately bad argument (like a `note_id` that doesn't exist): the `observation` log entry reads `"Error from tool 'read_note': ..."`, and the model's *next* message reacts to that error (asking for a different ID, or telling the user the note wasn't found) instead of stating a made-up note's contents as fact.

Difference from Intermediate, and from Approach 1's earlier form: Intermediate's version treats every tool result as a success. This Approach checks `result.isError` explicitly and labels a tool's *self-reported* failure clearly, the same way Doc06 taught for a Python exception's error message — the model gets a real chance to react correctly to a failure it can actually see, instead of reading error text with no signal that it *was* an error. It also warns (not fails) on a missing tool description, and replaces the plain `RuntimeError` with a named `MaxIterationsExceeded` carrying the log, so calling code can catch it specifically and still inspect what happened.

Which one should you actually write? Advanced Approach 1's version — treat `isError` handling and a named `MaxIterationsExceeded` as non-negotiable the moment tools come from a server you don't control, since a discovered tool failing in a way your bridge code silently swallows is exactly the kind of bug that's invisible until a demo goes wrong in front of someone. Basic's plain `while True` version is fine as a five-minute proof that the wiring works, but replace it with Intermediate's typed, limited loop before building anything else on top of it — Step 3 and Step 4 both extend this exact loop, so its shape needs to be right now.